`constexpr` union lifetime

Document #:	P3074R0
Date:	2023-12-15
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

Consider the following example:

```
template <typename T, size_t N>
struct FixedVector {
    union U { constexpr U() { } constexpr ~U() { } T storage[N]; };
    U u;
    size_t size = 0;

    // note: we are *not* constructing storage
    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(u.storage, u.storage+size);
    }

    constexpr auto push_back(T const& v) -> void {
        std::construct_at(u.storage + size, v);
        ++size;
    }
};

constexpr auto silly_test() -> size_t {
    FixedVector<std::string, 3> v;
    v.push_back("some sufficiently longer string");
    return v.size;
```

```
}
static_assert(silly_test() == 1);
```

This is basically how any static/non-allocating/in-place vector is implemented: we have some storage, that we *definitely do not value initialize* and then we steadily construct elements into it.

The problem is that the above does not work (although there is implementation divergence - MSVC and EDG accept it and GCC did accept it even up to 13.2, but GCC trunk and Clang reject).

Getting this example to work would allow `std::inplace_vector` ([P0843R9]) to simply work during `constexpr` time for all times (instead of just trivial ones), and was a problem briefly touched on in [P2747R0].

There are basically three ways we can approach this problem.

1.1 A library type: `std::uninitialized<T>`

We could introduce another magic library type, `std::uninitialized<T>`, with an interface like:

```
template <typename T>
class uninitialized {
    T storage; // exposition only

public:
    constexpr auto ptr() -> remove_extent_t<T>*;
    constexpr auto ptr() const -> remove_extent_t<T> const*;

    constexpr auto ref() -> remove_extent_t<T>&;
    constexpr auto ref() const -> remove_extent_t<T> const&;
};
```

As basically a better version of `std::aligned_storage`. Here is storage for a `T`, that implicitly begins its lifetime if `T` is an implicit-lifetime-type, but otherwise will not actually initialize it for you - you have to do that yourself. Likewise it will not destroy it for you, you have to do that yourself too.

`std::inplace_vector<T, N>` would then have a `std::uninitialized<T[N]>` and go ahead and `std::construct_at` (or, with [P2747R1], simply placement-new) into the appropriate elements of that array and everything would just work.

1.2 Just Make It Work

We could change the union rules such that if the first alternative of a `union` is an implicit-lifetime type, then its lifetime is started when the `union`'s lifetime is started. This is a pretty reasonable rule in my opinion, and follows from what implicit-lifetime means, and also seems to follow what the expectation might actually be for the above code.

As a result, the above example would just work with no further code changes, since the lifetime of `storage` is started (`T[N]` is an implicit-lifetime type for all `T`), which makes it the active member of the union, and we're all good on that front.

§ 1.3 Make the user explicitly start lifetime

One issue with just making it work, as described above, is what if you have something like:

```
union U {
    T x[N];
    U y[M];
} u;
```

Now what? Maybe in different contexts we want to populate `u.x` or `u.y`, and we can't implicitly start both alternatives' lifetimes. We have to choose.

To that end, we already have a seemingly-relevant function in the standard library:

```
template<class T>
  T* start_lifetime_as(void* p) noexcept;
```

Now, there are two problems with this function as far as its uses in `constexpr` go. The simple one is that it's not marked `constexpr`. The more complicated one is that the *Effects* of this function are:

> 3 — *Effects*: Implicitly creates objects ([intro.object]) within the denoted region consisting of an object a of type `T` whose address is `p`, and objects nested within `a`, as follows: The object representation of `a` is the contents of the storage prior to the call to `start_lifetime_as`. The value of each created object `o` of trivially copyable type ([basic.types.general]) `U` is determined in the same manner as for a call to `bit_cast<U>(E)` ([bit.cast]), where `E` is an lvalue of type `U` denoting `o`, except that the storage is not accessed. The value of any other created object is unspecified.

We can't really be talking about `bit_cast`ing anything out of our not-yet-even-initialized storage. That wording would have to change. But we could just say that during constant evaluation, this function simply starts the lifetime of the denoted object.

That is:

```
template <typename T, size_t N>
struct FixedVector {
    union U { constexpr U() { } constexpr ~U() { } T storage[N]; };
    U u;
    size_t size = 0;

    // note: we are *not* constructing storage
    constexpr FixedVector() {
        if consteval {
            std::start_lifetime_as<T[N]>(&u.storage);
        }
    }
};
```

This is a mildly inconvenient interface, since we have to repeat the type, but it has to match anyway. Plus we really don't want implementations to actually be copying anything in debug builds for sanitizing purposes, that would be completely wrong here - hence the `if consteval`. But also surely the only

reason to call `start_lifetime_as<T>(p)` is to actually then use the resulting `T*`, so implementations will presumably mark this function `[[nodiscard]]`.

It'd be annoying to introduce a new function (whose name would surely be similar) to achieve a similar feat, but we could do that:

```
template<class T>
  constexpr void start_lifetime(T*);
```

as in:

```
template <typename T, size_t N>
struct FixedVector {
    union U { constexpr U() { } constexpr ~U() { } T storage[N]; };
    U u;
    size_t size = 0;

    // note: we are *not* constructing storage
    constexpr FixedVector() {
        std::start_lifetime(&u.storage);
    }
};
```

This would be a function that would start the lifetime of the provided union alternative without performing any initialization. Which is the desired behavior here: it would simply require slightly more typing than the just make it work option.

Note that this would make implementing `std::uninitalized<T>` fairly straightforward - you just call the function if you need to (if `T` is trivially default constructible, you wouldn't need to).

§ 2 Proposal

This paper proposes the third option: introduce a new library function:

```
template<class T>
  constexpr void start_lifetime(T*);
```

Not to be confused with:

```
template<class T>
  /* not constexpr */ T* start_lifetime_as(void*);
```

Whose job it is to begin the lifetime of union alternative.

§ 2.1 Wording

Add to 20.2.2 [memory.syn]:

```
namespace std {
  // ...
  // [obj.lifetime], explicit lifetime management
```

```
  template<class T>
    T* start_lifetime_as(void* p) noexcept;
        // freestanding
  template<class T>
    const T* start_lifetime_as(const void* p) noexcept;
        // freestanding
  template<class T>
    volatile T* start_lifetime_as(volatile void* p) noexcept;
        // freestanding
  template<class T>
    const volatile T* start_lifetime_as(const volatile void* p) noexcept;
        // freestanding
  template<class T>
    T* start_lifetime_as_array(void* p, size_t n) noexcept;
        // freestanding
  template<class T>
    const T* start_lifetime_as_array(const void* p, size_t n) noexcept;
        // freestanding
  template<class T>
    volatile T* start_lifetime_as_array(volatile void* p, size_t n)
        noexcept;       // freestanding
  template<class T>
    const volatile T* start_lifetime_as_array(const volatile void* p,
        // freestanding
                                          size_t n) noexcept;

+ template<class T>
+   constexpr void start_lifetime(T* p) noexcept;
        // freestanding
}
```

With corresponding wording in 20.2.6 [obj.lifetime]:

```
template<class T>
  constexpr void start_lifetime(T* p) noexcept;
```

9 — *Mandates*: `T` is a complete type and an implicit-lifetime type.

10 — *Preconditions*: `p` is a pointer to a variant member of a union.

11 — *Effects*: Begins the lifetime ([basic.life]) of the non-static data member denoted by `p`. It is now the active member of its union. This ends the lifetime of the previously-active member of the union, if any.

§ 3 References

[P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. inplace_vector.
https://wg21.link/p0843r9

[P2747R0] Barry Revzin. 2022-12-17. Limited support for constexpr void*.
https://wg21.link/p2747r0

[P2747R1] Barry Revzin. 2023. `constexpr`` placement new.
https://wg21.link/p2747r1